

POWERTRACK: Etapas del SDLC

Presentado por: Juan Pablo Doria Caviedes



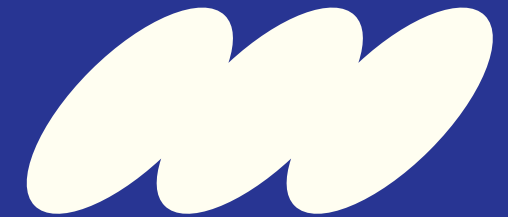
¿Qué es?

SDLC



- SDLC (Software Development Life Cycle)
- Ciclo de vida utilizado para planificar, desarrollar, probar, implementar y mantener un sistema de software.
- Proporciona una estructura metodológica que organiza cada etapa del desarrollo para garantizar orden, control y eficiencia en el proyecto.
- Permite definir tareas, responsabilidades, tiempos y recursos necesarios durante todo el proceso de creación del software.

Objetivos



- Garantizar la calidad del software mediante pruebas, validaciones y buenas prácticas de desarrollo.
- Reducir errores y riesgos durante el ciclo de vida del proyecto.
- Mantener el control de costos, tiempos y recursos para evitar retrasos y sobrecostos.
- Facilitar el mantenimiento y la actualización futura del sistema desarrollado.

Las 7 Fases



Planificación

Análisis de viabilidad técnica.



Análisis

Requisitos funcionales y no funcionales.



Diseño

Arquitectura y modelos de datos.



Implementación

Codificación basada en el diseño.

Pruebas

verificas que el código no falle y realmente haga lo que se pidió. hacer pruebas unitarias, de integración y buscar errores para que no le explote en la cara al usuario.

Despliegue

sacar el software de tu máquina local y ponerlo a funcionar en el mundo real. Involucra configurar los servidores, bases de datos en la nube y subir la aplicación a producción para que los clientes finales puedan usarla.

Mantenimiento

El software no se termina cuando se entrega. En esta fase corriges los errores que los usuarios encuentran en producción, instalas parches de seguridad y agregas nuevas funciones para que el sistema no se vuelva obsoleto.

Importancia del Alcance del Proyecto

Definición clara para el éxito

Un alcance bien definido permite establecer con claridad los objetivos, límites, entregables y requisitos de un proyecto, evitando confusiones durante su desarrollo. Además, ayuda a prevenir problemas como cambios innecesarios, retrasos, sobrecostos y fallos en la planificación. Definir correctamente el alcance también facilita la organización de tareas, la asignación de recursos y la comunicación entre los integrantes del equipo, aumentando las probabilidades de éxito y cumplimiento de los resultados esperados.



Requisitos

Diferenciando los tipos de requisitos en SDLC

Funcionales

Los requisitos funcionales definen las acciones específicas que el sistema debe realizar. Por ejemplo, iniciar sesión con correo y contraseña es un requisito funcional clave para el acceso de usuarios.

No Funcionales

Los requisitos no funcionales describen las características de rendimiento del sistema. Por ejemplo, tiempos de respuesta menores a dos segundos son fundamentales para garantizar una buena experiencia del usuario.

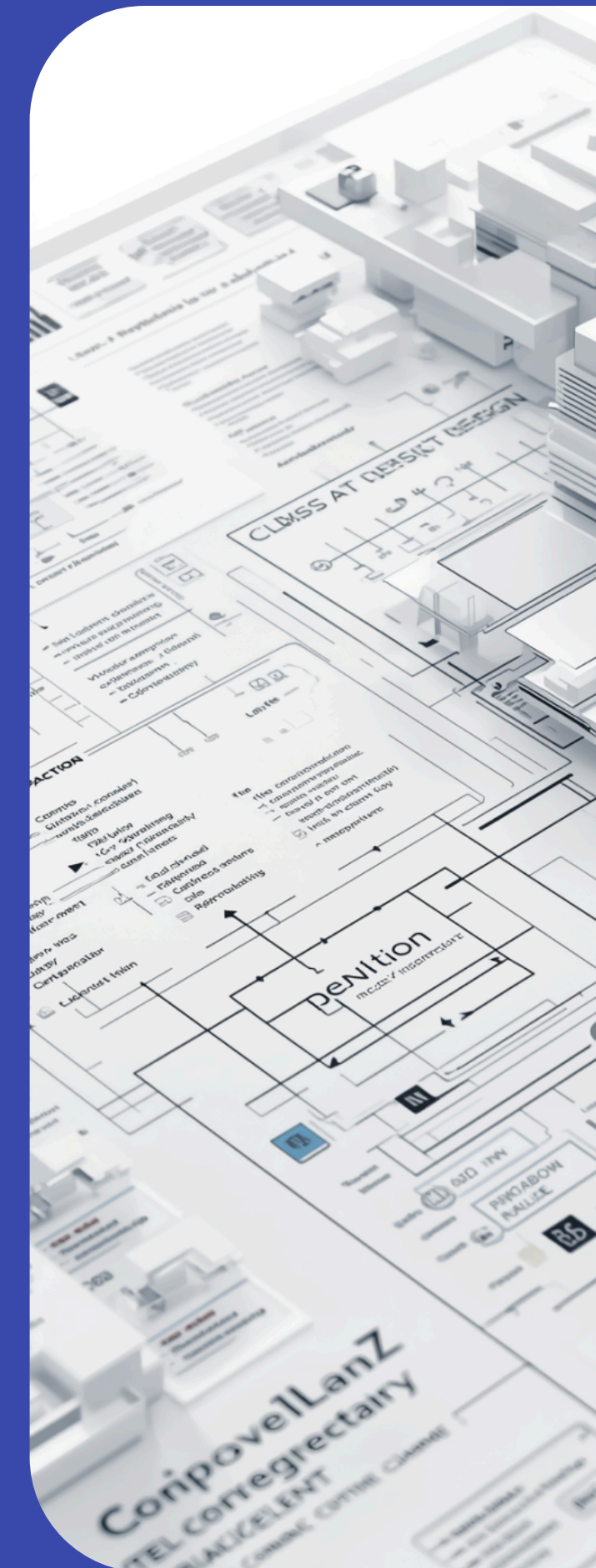
Importantes para el Éxito

Ambas categorías son vitales para el éxito del proyecto. Un buen balance entre requisitos funcionales y no funcionales asegura que el software cumpla con las expectativas y necesidades del usuario final.

Diseño HLD y LLD

Comparativa entre Alto y Bajo Nivel

El diseño de alto nivel (HLD, High-Level Design) y el diseño de bajo nivel (LLD, Low-Level Design) son etapas fundamentales en el desarrollo de software, ya que permiten estructurar y organizar correctamente un sistema antes de su implementación. El HLD se enfoca en definir la arquitectura general del sistema, incluyendo módulos, tecnologías, bases de datos, interacción entre componentes y flujo general de información. Por otro lado, el LLD se encarga de detallar la implementación específica de cada componente, describiendo algoritmos, estructuras de datos, clases, funciones y lógica interna del sistema. La combinación de ambos diseños facilita un desarrollo más ordenado, escalable, mantenible y eficiente.



Patrón MVC y Clean Code



Modelo

Lógica de negocio y manejo de datos



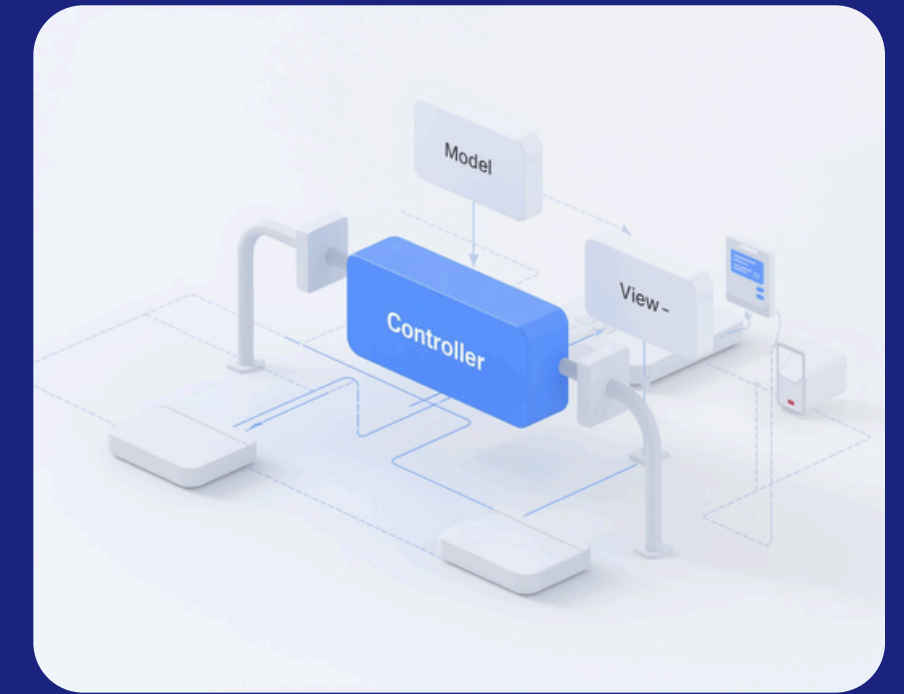
Vista

Interfaz de usuario y experiencia visual



Controlador

Gestión del flujo y lógica de aplicación



Clean Code

Código legible y mantenible por todos



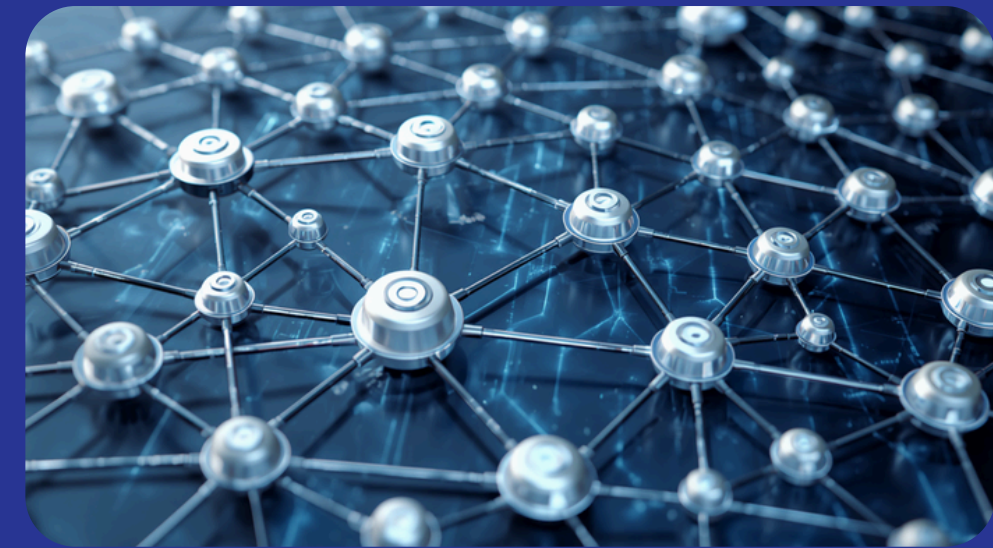
Pruebas Unitarias

Validan funciones y métodos individuales.



CI/CD

Automatización del pipeline de construcción.



Docker

Empaqueta aplicaciones en contenedores.



Mocks y Stubs

Simulan dependencias externas efectivamente.



Pruebas de Estrés

Simulan miles de usuarios simultáneos.



Validaciones Automáticas

Detectan errores rápidamente en el proceso.

Mantenimiento de Software

Estrategias para asegurar la calidad continua

Correctivo

El **mantenimiento correctivo** se enfoca en resolver defectos críticos en producción, como errores en POWERTRACK que afectan la experiencia del usuario o la integridad del sistema.

Evolutivo

El **mantenimiento evolutivo** implica agregar nuevas funcionalidades basadas en las necesidades del negocio, como la integración de un módulo de reservas en POWERTRACK para mejorar la oferta a los clientes.

Importancia

Ambos tipos de mantenimiento requieren un enfoque riguroso en las **pruebas de regresión** para garantizar que las nuevas funcionalidades y correcciones no introduzcan nuevos problemas en el software.

¿Qué herramienta permite empaquetar aplicaciones en contenedores?

Una de las herramientas más utilizadas para empaquetar aplicaciones en contenedores es Docker. Esta plataforma permite crear, distribuir y ejecutar aplicaciones dentro de contenedores, garantizando que funcionen de manera consistente en diferentes entornos y sistemas operativos.

Explica la diferencia entre mantenimiento correctivo y mantenimiento evolutivo.

El mantenimiento correctivo se realiza para solucionar errores, fallos o defectos detectados en el software después de su implementación. Su objetivo principal es restaurar el funcionamiento correcto del sistema.

Por otro lado, el mantenimiento evolutivo consiste en agregar nuevas funcionalidades, mejoras o adaptaciones al software para responder a nuevas necesidades del usuario o cambios tecnológicos. Este tipo de mantenimiento busca evolucionar y optimizar el sistema con el paso del tiempo.

15. Imagina que una aplicación bancaria falla porque no soporta muchos usuarios conectados al tiempo

15. Imagina que una aplicación bancaria falla porque no soporta muchos usuarios conectados al tiempo.

- **¿Qué tipo de requisito falló?**
- **Falló un requisito no funcional, específicamente el relacionado con el rendimiento y la escalabilidad del sistema, ya que la aplicación no soporta una alta cantidad de usuarios simultáneos.**
- **¿En qué fase del SDLC debió detectarse ese problema?**
- **Este problema debió detectarse principalmente en la fase de pruebas del SDLC, especialmente durante las pruebas de rendimiento, carga y estrés, donde se evalúa la capacidad del sistema para soportar múltiples usuarios y altos niveles de tráfico sin fallar.**